Weighted graph Voronoi cells

Algorithms of `voronoi.py`

Network Expansion Planning – EDF project

November 3, 2026

Abstract

The first step of the Voronoi + Steiner heuristic splits the nodes of the network between the sources (substations). Each node is assigned to the source that reaches it at the smallest *scaled* cost, where every edge length on a path starting at source s is divided by a weight w_s of that source. This document gives the algorithms of `voronoi.py` in pseudocode: the weighted multi-source Dijkstra, the construction of the cells, the splitting of the edges lying between two cells, and the shortest-path trees. It also states the properties these algorithms guarantee, then works through a small example.

1 Setting and notation

- $G = (V, E)$ is an undirected graph. Each edge (u, v) has a length $\ell(u, v) \geq 0$ (attribute `weight`, in km). Each node has a `label`, a position $\text{pos}(v) \in \mathbb{R}^2$ and a power `conso_ete`.
- $S = \{v \in V : \text{label}(v) = \texttt{Source}\}$ is the set of sources.
- Each source s has a weight $w_s > 0$ (Algorithm 1). Every edge on a path that starts at s costs $\ell(u, v)/w_s$. A larger weight makes s look closer, so its cell grows.
- $N(v)$ is the set of neighbours of v in G.

The algorithms return, for each node v, its scaled distance $\text{dist}(v)$, its source $\sigma(v) \in S$ and a path $\text{paths}(v)$ from $\sigma(v)$ to v. The cell of a source s is $C_s = \{v : \sigma(v) = s\}$.

Table 1 links each algorithm to its function in the code.

Algorithm	Name	Python function
1	SourceWeight	`source_weight`
2	DijkstraMultisourceSB	`_dijkstra_multisource_SB`, `multi_source_dijkstra_SB` (wrapper)
3	VoronoiCellsSB	`voronoi_cells_SB`
	CellsFromPaths	`cells_from_paths`
4	VoronoiCellsEdges	`voronoi_cells_edges`
5	EdgesOnShortestPaths	`edges_on_shortest_paths`
6	ComputeVoronoiCells	`compute_voronoi_cells`

Table 1: Algorithms of this document and the corresponding functions of `voronoi.py`.

2 Source weights

Four weightings are available. Three are fixed: `none`, `capacity` and `sqrt`. The fourth is a dictionary of free weights, which the variable-distance method (`voronoi_variable_distance.py`) adjusts iteratively.

Algorithm 1 Weight of a source

```
1: function SourceWeight(G, s, weighting)
2:     if weighting is a dictionary then
3:         return weighting[s]                                   ▷ free weights
4:     if weighting = none then
5:         return 1
6:     c ← conso_ete(s)                  ▷ capacity of the source, must be > 0
7:     if weighting = capacity then
8:         return c
9:     if weighting = sqrt then
10:        return √c
11:    error "unknown weighting"
```

3 Weighted multi-source Dijkstra

This is the core of the method. It is the multi-source Dijkstra of `networkx` with one change: when a settled node v relaxes its neighbours, the cost of each edge is divided by the weight of v's source, and an improved neighbour inherits that source.

Algorithm 2 Multi-source Dijkstra with costs scaled by the source weight

Input: graph G, non-empty set of sources $S \subseteq V$, weighting
Output: dist: scaled distance of each reached node; paths: path from its source

```
1: function DijkstraMultisourceSB(G, S, weighting)
2:     dist ← ∅;  paths ← ∅
3:     seen ← ∅                                          ▷ best tentative distances
4:     owner ← ∅                                     ▷ tentative source of each node
5:     H ← empty min-heap of triples (d, c, v);  c ← 0        ▷ counter c breaks ties
6:     for all s ∈ S do
7:         seen(s) ← 0;  owner(s) ← s;  paths(s) ← [s]
8:         push (0, c, s) on H;  c ← c + 1
9:     while H ≠ ∅ do
10:        (d, ·, v) ← pop the minimum of H
11:        if v ∈ dist then
12:            continue                          ▷ stale entry: v already settled
13:        dist(v) ← d                  ▷ v is settled: d and owner(v) are final
14:        s ← owner(v);  w_s ← SourceWeight(G, s, weighting)
15:        for all u ∈ N(v) do
16:            δ ← dist(v) + ℓ(v, u)/w_s          ▷ cost scaled by the weight of v's source
17:            if u ∈ dist then
18:                if δ < dist(u) then
19:                    error "contradictory paths: negative weights?"
20:            else if u ∉ seen or δ < seen(u) then          ▷ strict: ties keep the first owner
21:                seen(u) ← δ;  owner(u) ← s
22:                paths(u) ← paths(v) + [u]
23:                push (δ, c, u) on H;  c ← c + 1
24:    return dist, paths
```

The wrapper `multi_source_dijkstra_SB` first checks that $S \neq \emptyset$ and $S \subseteq V$, then calls Algorithm 2.

Properties. Write $\sigma(v) = \text{paths}(v)[0]$ for the source of a settled node v, and $C_s = \{v : \sigma(v) = s\}$.

Proposition 1 (Settling order)**.** *Nodes are settled in nondecreasing order of* dist*. If all lengths are nonnegative, the error on the contradictory-paths line is never raised.*

Proof. Every pushed key is $\text{dist}(v) + \ell(v,u)/w_s \geq \text{dist}(v)$, and v is the current minimum of the heap, so keys pushed later are never below those already popped. If u was settled before v, then $\text{dist}(u) \leq \text{dist}(v) \leq \delta$. □

Proposition 2 (Connected cells)**.** *For every settled v, every node of* paths(v) *lies in $C_{\sigma(v)}$. So each cell C_s is connected and contains s, and the union of the paths of a cell is a tree rooted at s.*

Proof. paths(u) is set to paths(v) + $[u]$ at the same moment as owner$(u) \leftarrow$ owner(v), and owner(v) is already final. By induction on the settling order, every prefix of paths(u) is the path of a node of the same cell. The paths are nested prefixes, so their union is a tree. □

Proposition 3 (What dist is)**.** *For every settled v,*

$$\text{dist}(v) = \frac{d_{G[C_{\sigma(v)}]}\big(\sigma(v), v\big)}{w_{\sigma(v)}},$$

where $d_{G[C]}$ is the shortest-path distance in the subgraph induced by C. Moreover, for every source s' and every path P from s' to v whose nodes, v excepted, all lie in $C_{s'}$, we have $\text{dist}(v) \leq \ell(P)/w_{s'}$.

Sketch. Inside a cell all edges are scaled by the same factor $1/w_s$, so the usual Dijkstra argument applies to $G[C_s]$. For the second claim, each node x of P is settled with owner s' and relaxes its successor on P with cost $\text{dist}(x) + \ell/w_{s'}$. By induction, $\text{dist}(v) \leq \ell(P)/w_{s'}$. □

Remark. This differs from the classical multiplicatively weighted Voronoi diagram, where v would go to $\arg\min_s d_G(s,v)/w_s$. Here a source cannot extend a path through a node another source has already settled. As a result the cells are always connected (Proposition 2), which the Steiner step needs. In the classical diagram, the cell of a strong source can surround a weak one or be cut into several pieces.

Complexity. Each edge is relaxed at most twice and each relaxation pushes at most one heap entry. With a binary heap, the running time is $O\big((|V| + |E|)\log|V|\big)$, like standard Dijkstra. The paths paths(u) = paths(v) + $[u]$ are copied explicitly, which costs $O(|V| \cdot h)$ memory, where h is the largest number of nodes on a path.

4 Voronoi cells

Algorithm 3 Voronoi cells from the shortest paths

```
1:  function VoronoiCellsSB(G, S, weighting)
2:      dist, paths ← DijkstraMultisourceSB(G, S, weighting)
3:      cells ← CellsFromPaths(G, paths)
4:      return dist, cells, paths

5:  function CellsFromPaths(G, paths)
6:      cells ← ∅                                   ▷ source ↦ set of nodes
7:      for all (v, p) ∈ paths do
8:          cells[p[0]] ← cells[p[0]] ∪ {v}          ▷ first node of the path is the source
9:      R ← V \ keys(paths)                         ▷ nodes reached by no source
10:     if R ≠ ∅ then
11:         cells[unreachable] ← R
12:     return cells
```

5 Splitting the edges between two cells

An edge (u, v) of length ℓ whose ends lie in two different cells is cut at the point q that is equidistant from both sources in the scaled metric. Write $w_u = w_{\sigma(u)}$ and $w_v = w_{\sigma(v)}$, and place q at distance $t\ell$ from u, with $t \in [0, 1]$. The condition

$$\operatorname{dist}(u) + \frac{t\,\ell}{w_u} \;=\; \operatorname{dist}(v) + \frac{(1-t)\,\ell}{w_v}$$

gives

$$t \;=\; \frac{\big(\operatorname{dist}(v) - \operatorname{dist}(u)\big)\,\dfrac{w_u w_v}{\ell} + w_u}{w_u + w_v} \qquad (\ell \neq 0), \tag{1}$$

and the code takes $t = 1$ when $\ell = 0$.

Proposition 4. *If* dist *comes from Algorithm 2 with the same weighting, then* $t \in [0, 1]$ *for every boundary edge.*

Proof. When v was settled, it relaxed u with cost $\operatorname{dist}(v) + \ell/w_v$. If u was still unsettled, u's final distance is at most that cost. If u was already settled, $\operatorname{dist}(u) \le \operatorname{dist}(v)$ by Proposition 1. Either way, $\operatorname{dist}(u) - \operatorname{dist}(v) \le \ell/w_v$, which is equivalent to $t \ge 0$ in (1). Exchanging the roles of u and v gives $\operatorname{dist}(v) - \operatorname{dist}(u) \le \ell/w_u$, which is equivalent to $t \le 1$. □

The code still checks $t \in [-10^{-9}, 1 + 10^{-9}]$ and then clamps t to $[0, 1]$. A larger deviation means that the cells and the distances were computed with different weightings.

Algorithm 4 Assign the edges to the cells and split the boundary edges (G is modified)

Input: cells without the key `unreachable`; dist from the same weighting; $V = \{0, \dots, |V|-1\}$
Output: edges of each cell, the split nodes, the distance of each split node to both sources

function VoronoiCellsEdges(G, *cells*, dist, *weighting*)
  if `unreachable` $\in$ *cells* **then**
    error "some nodes are reachable from no source"
  $E_k \leftarrow [\,]$ for each cell k; $E_{\texttt{split}} \leftarrow [\,]$
  $\text{cell}(n) \leftarrow k$ for each cell k and each $n \in \mathit{cells}[k]$
  $\mathit{next} \leftarrow |V|$ ▷ ids of the new nodes
  $\mathit{toAdd} \leftarrow [\,]$; $D \leftarrow \emptyset$
  for all $(u, v) \in E$ with length ℓ **do**
    if $\text{cell}(u) = \text{cell}(v)$ **then**
      append (u, v) to $E_{\text{cell}(u)}$ ▷ inner edge
      continue
    $w_u \leftarrow$ SourceWeight($G, \text{cell}(u), \mathit{weighting}$)
    $w_v \leftarrow$ SourceWeight($G, \text{cell}(v), \mathit{weighting}$)
    if $\ell \neq 0$ **then**
      $t \leftarrow \Big(\big(\text{dist}(v) - \text{dist}(u)\big)\, w_u w_v / \ell + w_u\Big) / (w_u + w_v)$ ▷ equation (1)
    else
      $t \leftarrow 1$
    if $t < -10^{-9}$ **or** $t > 1 + 10^{-9}$ **then**
      error "split point outside the edge"
    $t \leftarrow \min(\max(t, 0), 1)$
    $q \leftarrow \mathit{next}$; $\mathit{next} \leftarrow \mathit{next} + 1$
    $\text{pos}_q \leftarrow \text{pos}(u) + t\,\big(\text{pos}(v) - \text{pos}(u)\big)$ ▷ linear interpolation
    append $(q, \text{pos}_q, u, v, t\ell, (1-t)\ell)$ to *toAdd*
    append (u, q) to $E_{\text{cell}(u)}$; append (v, q) to $E_{\text{cell}(v)}$ ▷ one half-edge per cell
    append (u, v) to $E_{\texttt{split}}$
    $D(q) \leftarrow \big\{\text{cell}(u) : \text{dist}(u) + t\ell/w_u,\ \ \text{cell}(v) : \text{dist}(v) + (1-t)\ell/w_v\big\}$ ▷ equal values
  for all $(q, \text{pos}_q, u, v, l_1, l_2) \in \mathit{toAdd}$ **do** ▷ after the loop over E
    add node q to G with label `dummy` and position pos_q
    add edges (u, q) of length l_1 and (v, q) of length l_2, label `dummy` ▷ (u, v) is kept
  return $(E_k)_k \cup \{E_{\texttt{split}}\},\ \mathit{toAdd},\ D$

6 Shortest-path trees of the cells

The edges used by the shortest paths of a cell form a tree rooted at its source (Proposition 2). This tree is the "Voronoi-only" network, before the Steiner step.

Algorithm 5 Edges on the shortest paths of each cell

function EdgesOnShortestPaths(*cells*, paths)
  for all cell k **do**
    $U_k \leftarrow \emptyset$
    for all $n \in \mathit{cells}[k]$ **do**
      $p \leftarrow \text{paths}(n)$
      for $i \leftarrow 0$ **to** $|p| - 2$ **do**
        $U_k \leftarrow U_k \cup \big\{(\min(p_i, p_{i+1}),\ \max(p_i, p_{i+1}))\big\}$
  return $\{k : \text{sorted}(U_k)\}$

7 Entry point

Algorithm 6 runs the whole pipeline and stores the results in the `GraphData` object. The original graph is not modified. The split edges are added to a copy, G_{split}, which is used only for plotting. The Steiner step that follows runs on the original G, using the node cells.

Algorithm 6 Voronoi cells of the sources of a network

procedure ComputeVoronoiCells(*graph*, *weighting*)
  $G \leftarrow graph.G$; $S \leftarrow \{n \in V : \text{label}(n) = \texttt{Source}\}$
  graph.dist, *graph.cells*, *graph*.paths $\leftarrow$ VoronoiCellsSB(G, S, *weighting*)
  graph.usedEdges $\leftarrow$ EdgesOnShortestPaths(*graph.cells*, *graph*.paths)
  $G_{\text{split}} \leftarrow$ copy of G
  cells $\leftarrow$ *graph.cells*; dist $\leftarrow$ *graph*.dist
  graph.edgeCells, *toAdd*, $\cdot \leftarrow$ VoronoiCellsEdges(G_{split}, *cells*, dist, *weighting*)
  graph.$G_{\text{split}} \leftarrow G_{\text{split}}$
  for all $(q, \text{pos}_q, u, v, \cdot, \cdot) \in$ *toAdd* **do** ▷ plotting only
    add point q and segments $u \to q$, $v \to q$ to copies of the node and edge tables

8 Worked example

Consider the path graph of Figure 1, with capacity weighting $w_{S_1} = 3$, $w_{S_2} = 1$ and all edge lengths equal to 3.

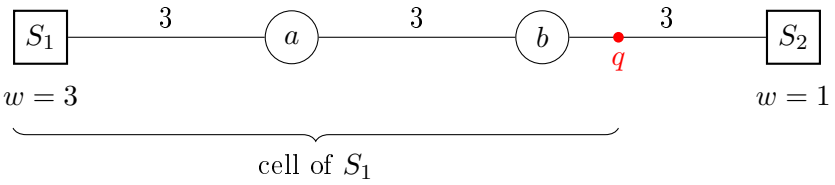


Figure 1: The source S_1 is three times stronger than S_2: its cell takes b, and the boundary point q is three times closer to S_2 than to S_1 (2.25 km against 6.75 km).

Table 2 traces Algorithm 2.

Step	Popped (owner)	Relaxations	Heap afterwards
1	S_1 (0)	a: $0 + 3/3 = 1$, owner S_1	$(0, S_2), (1, a)$
2	S_2 (0)	b: $0 + 3/1 = 3$, owner S_2	$(1, a), (3, b)$
3	a $(1, S_1)$	b: $1 + 3/3 = 2 < 3$, b moves to S_1	$(2, b), (3, b)$
4	b $(2, S_1)$	S_2 already settled, $2 + 1 \geq 0$, nothing to do	$(3, b)$
5	b (3)	stale entry, skipped	empty

Table 2: Run of the weighted Dijkstra on the example.

The result is $C_{S_1} = \{S_1, a, b\}$ and $C_{S_2} = \{S_2\}$: although b is closer to S_2 in plain kilometres, the stronger source takes it. The only boundary edge is (b, S_2), with $\ell = 3$, $\text{dist}(b) = 2$, $\text{dist}(S_2) = 0$, $w_u = 3$ and $w_v = 1$. Equation (1) gives

$$t = \frac{(0 - 2) \cdot 3 \cdot 1/3 + 3}{3 + 1} = \frac{1}{4},$$

so q is 0.75 km from b and 2.25 km from S_2. As a check, the scaled distance to q is $2 + 0.75/3 = 2.25$ from the S_1 side and $0 + 2.25/1 = 2.25$ from the S_2 side.